# Semgrep 代码分析实验报告

李琪轩 25140931

## 一、源码问题分析

在 code-repo/app.py 中，分析一个具体的问题。本次实验分析的是一个命令注入漏洞。

首先分析源码 source：

```python
@app.route("/diag/ssh", methods=["GET", "POST"])
def diag_ssh():
    if "user_id" not in session:
        return redirect(url_for("login"))
    output = None
    if request.method == "POST":
        host = request.form.get("host", "")
        username = request.form.get("username", "")
        password = request.form.get("password", "")
        try:
            client = paramiko.SSHClient()
            client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
            client.connect(host, username=username, password=password, timeout=5)
            _, stdout, _ = client.exec_command("whoami")
            output = stdout.read().decode()
            client.close()
        except Exception as e:
            output = f"Error: {e}"
    return render_template("diag.html", ssh_output=output)

```

数据来自用户前端表单输入，完全不可信。

分析 path：

用户输入的 host 变量未经过任何过滤、净化、校验，直接拼接到系统命令字符串中。

总结 sink：

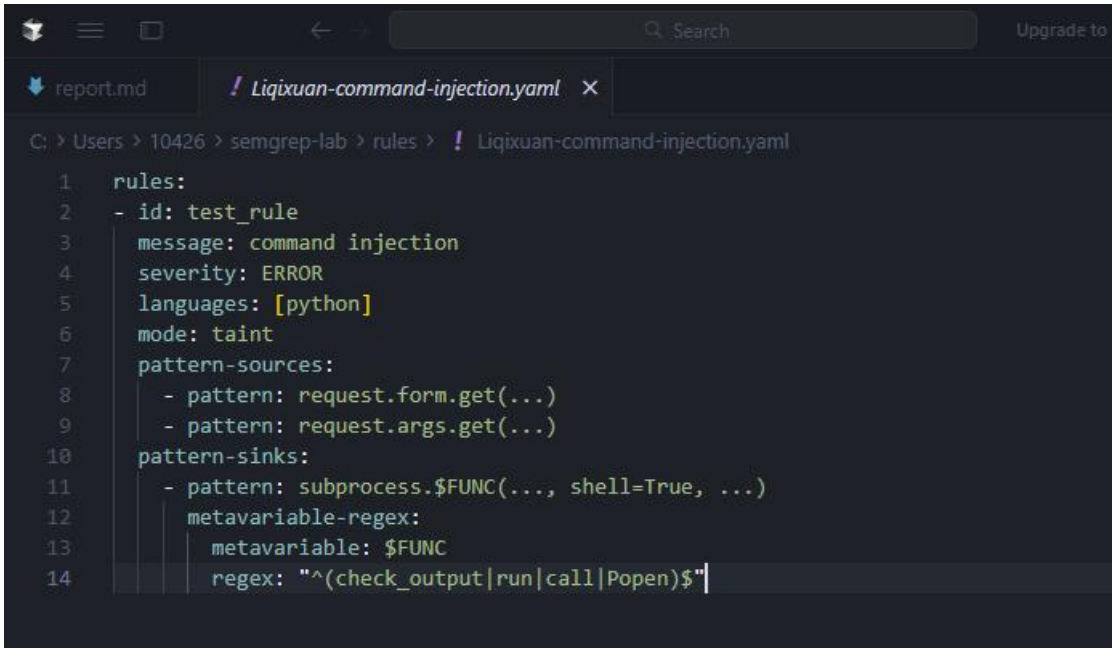
```python
@app.route("/diag/ping", methods=["GET", "POST"])
def diag_ping():
    if "user_id" not in session:
        return redirect(url_for("login"))
    output = None
    if request.method == "POST":
        host = request.form.get("host", "")
        # B602: subprocess with shell=True and unsanitized user input — command injection
        output = subprocess.check_output("ping -c 1 " + host, shell=True,
                                         stderr=subprocess.STDOUT).decode()
    return render_template("diag.html", output=output)

# ------------------------------------------------------------------------
```

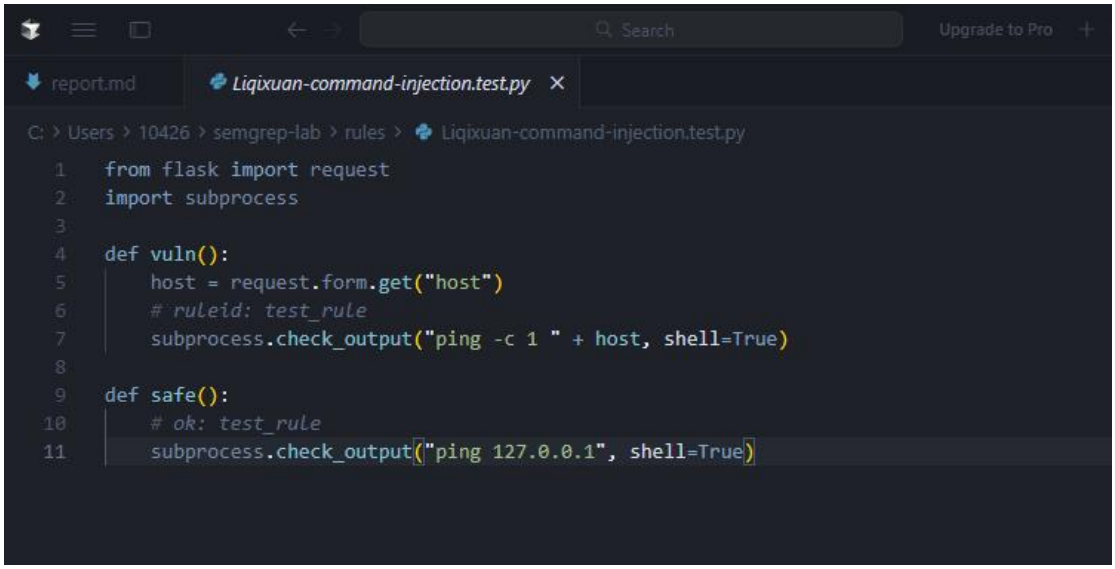
脏数据流入 subprocess 执行函数，且开启 shell=True，触发命令注入。

二、基于 semgrep 的规则编写

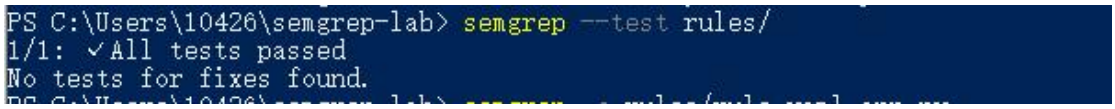
规则文件如下：




编写测试文件：

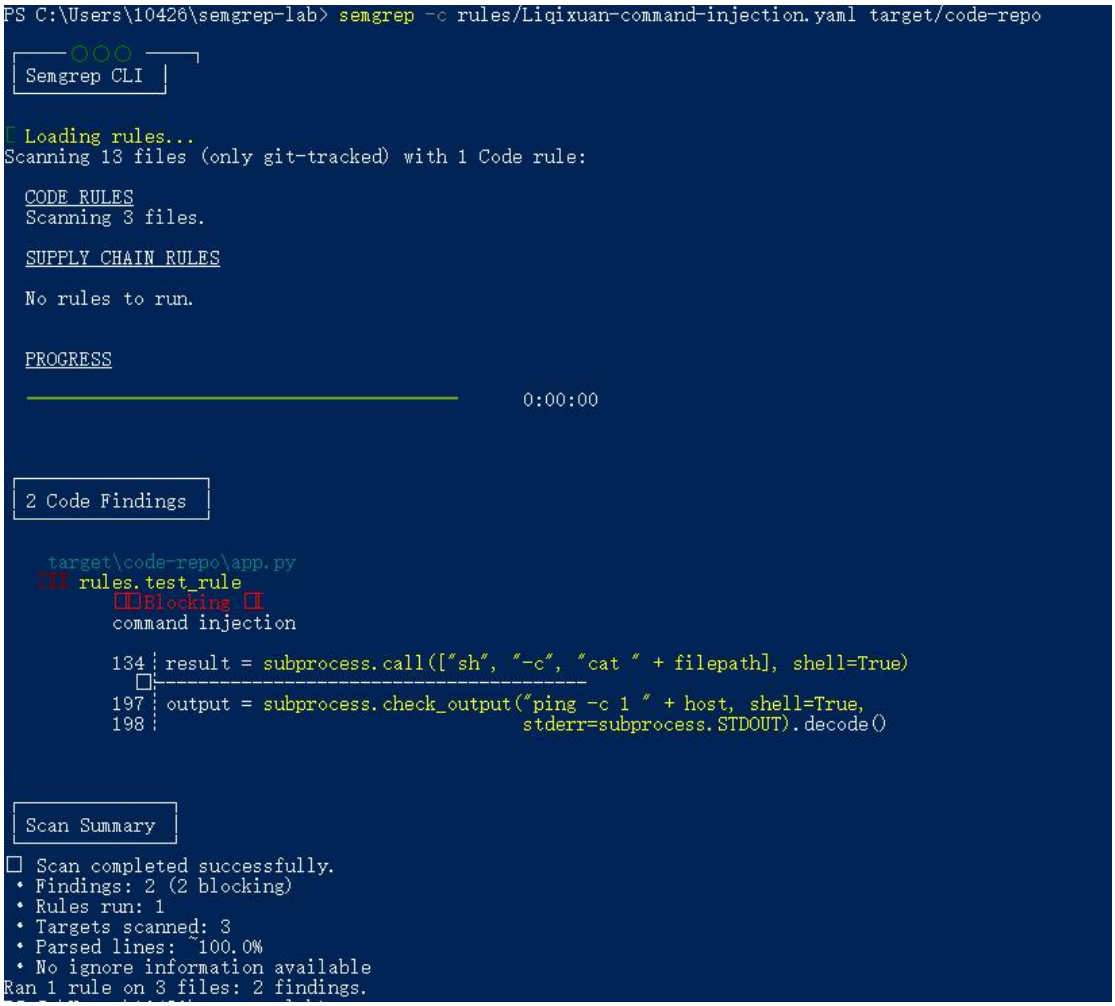



运行测试：




三、执行漏洞扫描

执行结果如下


PS C:\Users\10426\semgrep-lab> semgrep -c rules/Liqixuan-command-injection.yaml target/code-repo
Semgrep CLI
Loading rules...
Scanning 13 files (only git-tracked) with 1 Code rule:
CODE RULES
Scanning 3 files.
SUPPLY CHAIN RULES
No rules to run.
PROGRESS
0:00:00
2 Code Findings
target\code-repo\app.py
rules.test_rule
Blocking
command injection
134 result = subprocess.call(["sh", "-c", "cat " + filepath], shell=True)
197 output = subprocess.check_output("ping -c 1 " + host, shell=True,
198 stderr=subprocess.STDOUT).decode()
Scan Summary
Scan completed successfully.
• Findings: 2 (2 blocking)
• Rules run: 1
• Targets scanned: 3
• Parsed lines: ~100.0%
• No ignore information available
Ran 1 rule on 3 files: 2 findings.